

Findings — OmniRoute: integrazioni agentiche, pattern fallback, nodo ToS — 2026-07-25

Ricerca condotta dall'Argonauta Personale su brief di Dedalo (Architetto).

Obiettivo: dati grezzi sul meccanismo di integrazione di OmniRoute e router simili

(LiteLLM, OpenRouter) in sistemi multi-agente; gestione fallback + nodo ToS Claude.

Interpretazione e decisione progettuale restano a Dedalo.

1. Sistemi/progetti agentici reali che integrano questi router

OpenAI Agents SDK + LiteLLM (docs.litellm.ai/docs/projects/openai-agents)

Integrazione ufficiale documentata: ogni Agent istanzia un `LitellmModel` con `base_url="http://localhost:4000"` e riferenzia il nome del modello definito nel `config.yaml` centrale del proxy. Ogni agente può usare un modello diverso — l'orchestrazione della diversità avviene a livello di istanziamento Python, non di routing dinamico.

OmniRoute (github.com/diegousouzpw/OmniRoute)

Gateway locale, porta 20128, endpoint `/v1/chat/completions` OpenAI-compatibile. Supporta MCP (94 tool), A2A (JSON-RPC 2.0 + SSE), integrazione dichiarata con Claude Code, Codex, Cursor, Cline, Continue, Aider (33+ tool configurabili via `base_url: http://localhost:20128/v1`). 500+ contributor su GitHub. Rilascio attivo: v3.8.49 al 22 luglio 2026.

LiteLLM Proxy (docs.litellm.ai)

Il router self-hosted più usato in produzione per sistemi agentici. Config YAML centralizzata, endpoint su `localhost:4000`, RBAC per chiavi, osservabilità integrata.

Usato in tandem con OpenRouter: LiteLLM gestisce logging/RBAC locale, OpenRouter gestisce la breadth dei provider a monte.

projdevbench (deepwiki.com/zsworld6/projdevbench)

Esempio documentato di configurazione LiteLLM + OpenRouter con `ANTHROPIC_BASE_URL` e `OPENAI_BASE_URL` puntati al proxy locale. Pattern usato per sviluppo agente multi-modello.

Nessun sistema agentivo di terze parti (AutoGen, CrewAI, LangGraph) risulta avere un'integrazione nativa ufficiale con OmniRoute. L'integrazione avviene sempre via `base_url` redirect — non è framework-specifica.

2. Meccanismo tecnico di integrazione

Come si punta il router

Pattern uniforme in tutti i sistemi trovati:

```
...
```

```
OPENAI_BASE_URL=http://localhost:4000 # per LiteLLM
```

```
ANTHROPIC_BASE_URL=http://localhost:20128/v1 # per OmniRoute (se intercetta Anthropic)
```

```
...
```

Oppure, a livello di codice:

```
```python
```

## OpenAI Agents SDK + LiteLLM: per-agent model assignment

```
agente_orchestratore = Agent(
```

```
model=LitellmModel(model="gpt-4o", base_url="http://localhost:4000", api_key="sk-...")
```

```
)
```

```
agente_coder = Agent(
```

```
model=LitellmModel(model="claude-sonnet", base_url="http://localhost:4000", api_key="sk-...")
```

```
)
```

```
...
```

Il proxy riceve la richiesta, la trasforma nel formato del provider reale, la spedisce.

Il chiamante non vede il provider — vede solo il nome-modello definito nel config.

### Config YAML LiteLLM (pattern verificato dalla doc ufficiale)

```
```yaml
```

```
model_list:
```

```
- model_name: "gpt-4o"
```

```
litellm_params:
```

```
model: openai/gpt-4o
```

```
api_key: os.environ/OPENAI_API_KEY
```

```
- model_name: "claude-sonnet"
```

```
litellm_params:
```

```
model: anthropic/claude-sonnet-4
```

```
api_key: os.environ/ANTHROPIC_API_KEY
```

```

router_settings:
fallbacks: [{"gpt-4o": ["claude-sonnet"]}]]
context_window_fallbacks: [{"claude-sonnet": ["gpt-4o"]}]]
default_fallbacks: ["gpt-4o"]
...

```

Come scatta il fallback

LiteLLM implementa tre strati separati:

- **Provider circuit breaker** — se l'intero provider è down, rerouting immediato
- **Connection cooldown** — exponential backoff per chiave API specifica al rate-limit
- **Model lockout** — modello specifico flaggato fuori per finestra temporale

Tipi di fallback distinti:

- `fallbacks` — per `RateLimitError`, errori 429/500 generici
- `context_window_fallbacks` — pre-call, se il prompt supera il contesto del modello (richiede `enable_pre_call_checks: true`)
- `content_policy_fallbacks` — per violazioni di policy
- `default_fallbacks` — catch-all per il resto

Il fallback è trasparente al chiamante: il client riceve la risposta senza sapere

quale modello ha risposto. L'header `x-litellm-model-id` nella risposta identifica

il modello che ha effettivamente servito.

Routing per-agente/per-task

Pattern trovato (da zylos.ai/research, verificato):

```
```python
```

# routing interno all'agente per tipo di task

```

if task_type == "ragionamento_denso":
return await self.invoke(task, model="claude-opus-4")
elif task_type == "generazione_codice":
return await self.invoke(task, model="claude-sonnet-4")
else:
return await self.invoke(task, model="gpt-4o-mini")
...

```

Framework MasRouter (trovato in letteratura): tre livelli — collaboration determiner

(task singolo vs multi-agente), role allocator (che ruoli servono), LLM router

(quale modello per ogni ruolo). Non è un prodotto commerciale documentato, è un

pattern architetturale.

---

### 3. Il nodo ToS — dati certi

---

#### Cosa è successo (verificato su 4 fonti)

Febbraio 2026: Anthropic ha aggiornato i ToS vietando esplicitamente l'uso dei token OAuth delle subscription (Free, Pro, Max) fuori dai client ufficiali. Enforcement attivo da aprile 2026. Fonte principale: [winbuzzer.com/2026/02/19](https://winbuzzer.com/2026/02/19) + [gigazine.net](https://gigazine.net) + [dev.to/mcrolly](https://dev.to/mcrolly).

Citazione esatta (da documentazione Anthropic riportata): "OAuth tokens obtained with the Free, Pro, and Max plans are for use exclusively with Claude Code and Claude.ai, and use of these tokens with any other products, tools, or services, including the Agent SDK, is unauthorized and a violation of its consumer terms of service."

#### OmniRoute e il provider "Claude Web"

OmniRoute espone un provider chiamato "Claude Web" che usa cookie-session di [claude.ai](https://claude.ai) (6 cookie: `sessionKey`, `routingHint`, `cf_clearance`, `__cf_bm`, `_cfuid` + metadata routing).

La wiki di OmniRoute non contiene disclaimer ToS espliciti su questo provider.

La sostanza tecnica — autenticazione via cookie di sessione estratti da [claude.ai](https://claude.ai) — è funzionalmente equivalente al ban OAuth: usa credenziali di subscription al di fuori del client ufficiale. Rischio concreto: sospensione account.

OmniRoute cataloga "15 provider ToS-flagged" (auto-dichiarazione del progetto) e li segnala nel dashboard perché "l'utente decide". [iflytek/Sparkdesk](https://iflytek.com) e [Antigravity](https://antigravity.ai) risultano fra i flaggati. Non è confermato se "Claude Web" sia esplicitamente nel gruppo flagged — la wiki tratta il provider in modo tecnico senza avvertenze legali.

#### Cosa resta legale per usare Claude in sistemi agentici

- **Claude Code CLI** su macchina locale o server via SSH — esplicitamente permesso nei ToS, incluso in pipeline automatizzate e cron CI/CD
- **API key da Anthropic Console** — sempre consentita, costo a consumo (\$0.80/M input per Sonnet 4 secondo [kersai.com](https://kersai.com))
- **AWS Bedrock / Google Vertex AI** — provider cloud che offrono Claude via API, fuori dai ToS consumer, regolati da accordi enterprise

Fonte: [autonomee.ai/blog/claude-code-tos](https://autonomee.ai/blog/claude-code-tos) + [kersai.com/workarounds](https://kersai.com/workarounds)

---

## 4. Pattern concreti di fallback chain al rate-limit

---

### Cosa succede al contesto quando il modello cambia a metà

Dato critico trovato in [github.com/BerriAI/litellm/discussions/25703](https://github.com/BerriAI/litellm/discussions/25703):

"switching the horse mid-conversation is going to be non-trivial because frontier LLMs are increasingly particular about continuing existing conversation threads."

Il proxy passa il contesto originale al modello di fallback — tecnicamente funziona.

Ma il problema è semantico: un modello diverso ha stile, training, e risposte di sistema diversi. Il cambio a metà di una chain agentiva può produrre incoerenze sottili.

Non è un problema di infrastruttura; è un problema di continuità cognitiva del task.

La workaround documentata è architetturale: il routing per-task (non per-sessione)

è più sicuro del routing mid-session. Si assegna il modello al tipo di task prima che inizi, non si cambia strada facendo.

### Pattern OpenRouter per free tier al limite

Da [brainroad.com](https://brainroad.com) (verificato):

- Limite free OpenRouter: 50 richieste/giorno (shared fra tutti i modelli free) + 20 RPM; \$10 un-tantum eleva il daily a 1000 mantenendo 20 RPM
- Fallback chain consigliata per free tier: DeepSeek Chat v3 → Llama 4 Maverick → Qwen3-235b (unici tre che reggono tool-calling in modo affidabile sui test)
- Routing endpoint `openrouter/free` = auto-selezione fra tutti i free, non deterministica — non sai quale modello risponde

### OmniRoute: 4 tier di fallback dichiarati

Tier 1 Subscription → Tier 2 API Key → Tier 3 Cheap → Tier 4 Free

19 strategie di routing disponibili. Le principali rilevanti per uso agentivo:

- `priority` : drena quota del provider corrente poi passa al successivo
- `cost-optimized` : prezzi live, minimizza costo per request
- `auto` : 12-factor scoring su salute/quota/latenza/success-rate
- `fusion` : fan-out su N modelli + judge sintetizza (costoso ma robusto)
- `pipeline` : output sequenziale attraverso provider diversi (tipo chain-of-thought cross-model)

Tre layer di resilienza: circuit breaker provider, cooldown per chiave, lockout per modello specifico.

### Openai Agents SDK: integrazione LiteLLM router NON supportata nativamente

Issue #813 su [github.com/openai/openai-agents-python](https://github.com/openai/openai-agents-python): la classe `Router` di LiteLLM

non può essere passata direttamente come `model` a `Agent()` — solo `LitellmModel`.

Non è pianificato il supporto. Quindi: il fallback deve essere configurato lato proxy (LiteLLM server), non lato agente. L'agente punta al proxy, il proxy gestisce il fallback in modo trasparente. Questa è la separazione di responsabilità corretta.

---

## 5. Pattern riusabili per la navicella — dati grezzi (Dedalo decide)

---

Questi sono i pattern tecnici trovati, senza interpretazione architetturale.

### P1 — Base URL redirect per-agente

Ogni agente ha `base_url` separato che punta al proxy. Il proxy assegna il modello reale per nome. Cambio di modello = cambio del name nel config.yaml, non nel codice dell'agente. Zero refactoring agente per cambiare provider.

### P2 — Fallback chain dichiarativa in YAML

Separazione: il codice agente non sa dei fallback. Il proxy li gestisce. Configurazione in config.yaml: modello principale → fallback list → default fallback. Il modello di fallback riceve il contesto originale invariato.

### P3 — Free tier chain affidabile (se si vuole abbassare il costo agenti secondari)

DeepSeek V3 via SiliconFlow (gratis forever) o via OpenRouter (:free variant) per agenti leggeri (Rapsodo, Argonauta). I free tier reggono task semplici ma non task agentivi complessi multi-step. Limite: 50 req/giorno OpenRouter free, oppure ~2.4M token/giorno SiliconFlow free per DeepSeek V3.

### P4 — Claude in produzione agentiva: solo API key o Claude Code CLI

Subscription OAuth = ban dal febbraio 2026. Claude Web via cookie = stesso rischio. L'unica via per Claude in pipeline automatizzata è: (a) API key console.anthropic.com, (b) Claude Code CLI direttamente, (c) Bedrock/Vertex per contratti enterprise.

### P5 — Routing per-task vs routing mid-session

Il cambio modello a metà di una conversazione agentiva è tecnicamente supportato dai proxy ma semanticamente rischioso (incoerenza stile/comportamento). Pattern consolidato: assegna il modello per tipo di agente/task PRIMA che la sessione inizi, non cambiarlo mentre è attiva.

### P6 — Compressione token OmniRoute: RTK+Caveman

RTK filtra risultati di tool ridondanti. Caveman comprime prosa con rule-based stripping. LLMlingua-2 (opzionale, ML-based). Risparmio dichiarato: 89.2% su sessioni tool-heavy (dato auto-risportato, non validato da terzi). Rilevante per

la navicella se il contesto degli agenti cresce molto per sessione.

## **P7 — OmniRoute Claude Web: da evitare**

Usa cookie di sessione claude.ai estratti manualmente. Funzionalmente è un proxy della subscription, vietato dai ToS Anthropic. OmniRoute non ha disclaimer espliciti su questo nel wiki tecnico — è responsabilità dell'utente. Non adottare questo provider nella navicella.

---

## **Fonti verificate**

---

- [github.com/diegosouzapw/OmniRoute](https://github.com/diegosouzapw/OmniRoute) (repo principale + AGENTS.md + wiki Claude-Web)
- [docs.litellm.ai/docs/proxy/reliability](https://docs.litellm.ai/docs/proxy/reliability) (fallback types e config YAML)
- [docs.litellm.ai/docs/projects/openai-agents](https://docs.litellm.ai/docs/projects/openai-agents) (integrazione OpenAI Agents SDK)
- [docs.litellm.ai/docs/routing](https://docs.litellm.ai/docs/routing) (routing groups, ordering, config)
- [github.com/openai/openai-agents-python/issues/813](https://github.com/openai/openai-agents-python/issues/813) (LiteLLM Router non supportato in Agent())
- [github.com/BerriAI/litellm/discussions/25703](https://github.com/BerriAI/litellm/discussions/25703) (problema mid-conversation switch)
- [zylos.ai/research/2026-03-02-ai-agent-model-routing/](https://zylos.ai/research/2026-03-02-ai-agent-model-routing/) (pattern per-task routing, MasRouter, fallback chain config)
- [winbuzzer.com/2026/02/19/anthropic-bans-claude-subscription-oauth-in-third-party-apps-xcxwbn/](https://winbuzzer.com/2026/02/19/anthropic-bans-claude-subscription-oauth-in-third-party-apps-xcxwbn/)
- [gigazine.net/gsc\\_news/en/20260220-anthropic-third-party-block/](https://gigazine.net/gsc_news/en/20260220-anthropic-third-party-block/)
- [autonomiee.ai/blog/claude-code-terms-of-service-explained/](https://autonomiee.ai/blog/claude-code-terms-of-service-explained/)
- [kersai.com/anthropic-killed-third-party-claude-access-heres-every-workaround-that-still-works/](https://kersai.com/anthropic-killed-third-party-claude-access-heres-every-workaround-that-still-works/)
- [mindstudio.ai/blog/anthropic-openclaw-ban-oauth-authentication](https://mindstudio.ai/blog/anthropic-openclaw-ban-oauth-authentication)
- [brainroad.com/openrouter-free-models-which-ones-actually-work-for-ai-agents/](https://brainroad.com/openrouter-free-models-which-ones-actually-work-for-ai-agents/)
- [wavect.io/blog/llm-gateway-router-comparison-2026/](https://wavect.io/blog/llm-gateway-router-comparison-2026/)
- [aitoolly.com/ai-news/article/2026-07-21-omniroute-a-universal-mit-licensed-ai-gateway](https://aitoolly.com/ai-news/article/2026-07-21-omniroute-a-universal-mit-licensed-ai-gateway)
- [explainx.ai/blog/omniroute-ai-gateway-free-llm-proxy-claude-code-2026](https://explainx.ai/blog/omniroute-ai-gateway-free-llm-proxy-claude-code-2026)